VoltEdge 2.0 — Project Report

Executive Summary

VoltEdge 2.0 is an agentic circuit-design application that lets a user describe an electronics board in natural language, then uses a Claude Code–driven backend agent to create and iteratively update a tscircuit workspace. The system combines a FastAPI backend, a React/Vite frontend, the Claude Agent SDK, and the tscircuit toolchain to support a conversational circuit-building workflow with live build feedback and visual circuit preview.

The project has progressed through its de-risking phase and a working vertical slice. Phase 0 validated the main technical assumptions: tscircuit can be scaffolded and built headlessly, JLCPCB parts can be imported non-interactively, fabrication outputs can be exported as a Gerber/BOM/PnP zip, browser rendering is feasible, and the Claude Agent SDK can drive a real circuit-generation turn. Phase 1 then implemented the core create prompt stream build render loop with project persistence, session management, SSE event streaming, a frontend chat interface, and tscircuit preview integration.

At the current stage, VoltEdge is a functional local prototype rather than a finished production product. The core end-to-end loop exists, but later planned capabilities such as structured plan cards, question gating, full guardrail UX, export polish, idle-session lifecycle management, and complete test/CI coverage remain future work.

Project Motivation

Designing a simple PCB or electronics module usually requires switching between several specialized tools: schematic editors, PCB layout software, component databases, datasheets, fabrication export tools, and manual verification workflows. This creates a steep learning curve for users who know what circuit they want but do not want to manually operate a full EDA stack.

VoltEdge addresses this by turning circuit design into an interactive agent-assisted workflow. The user can describe the desired board in plain language, while the agent writes tscircuit source code, checks the build, updates the board, and streams its progress back to the interface. The goal is not to replace expert engineering review or manufacturability validation, but to make early circuit drafting, iteration, visualization, and export faster and more accessible.

Core Contribution

The project contributes a working architecture for a conversational circuit-design assistant that connects natural-language agent interaction with a real circuit compiler and browser-based circuit preview.

In one sentence: VoltEdge turns a natural-language board request into a persistent tscircuit project by routing Claude Agent SDK tool use through a guarded FastAPI backend and rendering the resulting circuit workspace in a React-based visual editor.

System Overview

VoltEdge is organized around four main surfaces:

- Frontend application
- React 19 and Vite application.
- Chat-first interface for user prompts and streamed agent events.
- Session sidebar for listing, opening, renaming, and deleting projects.
- Circuit preview pane powered by `@tscircuit/runframe`.
- Drag-to-edit support that persists component placement back into `index.circuit.tsx`.

- Backend service
- FastAPI application exposing project, message, placement, fsMap, event, and interrupt routes.
- SQLite/SQLModel persistence for projects, sessions, messages, checkpoints, and event history.
- SSE event stream for live agent progress.
- Workspace service for creating and repairing tscircuit workspaces.

- Agent runtime
- Claude Agent SDK session per active project.
- Project-specific working directory.
- tscircuit and components skills mounted into the agent workspace.
- Tool mapping from SDK messages to frontend-friendly SSE events.
- Bash allowlist and file-edit confinement for safer agent operation.

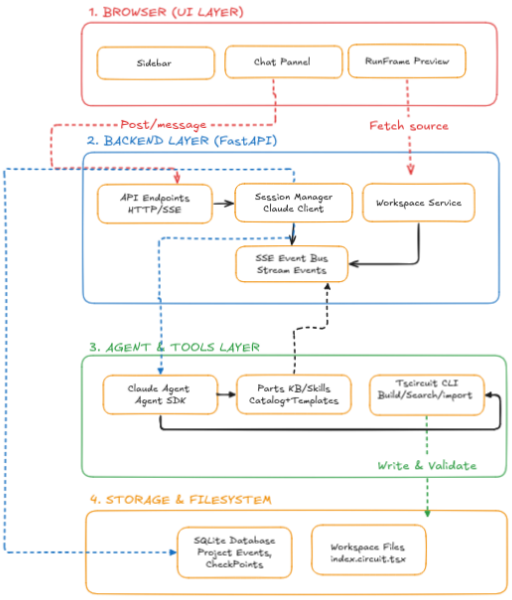
- Circuit toolchain
- `tsci` for workspace initialization, checking, building, importing parts, and exporting fabrication outputs.
- bun as a required runtime for the tscircuit CLI.
- Generated `dist/<entrypoint>/circuit.json` as the central build artifact.

Architecture

The system follows a browser-to-backend-to-agent architecture:

```
User
  │
  ▼
React/Vite frontend
  ├─ Chat panel: prompts, assistant text, thinking blocks, tool chips, errors
  ├─ Session sidebar: project navigation and management
  └─ Preview pane: tscircuit RunFrame schematic/PCB/CAD rendering
  │
  ▼ HTTP + SSE
FastAPI backend
  ├─ Project routes: create/list/rename/delete projects
  ├─ Message route: enqueue one agent turn per project
  ├─ Event route: stream SDK-derived events over SSE
  ├─ Workspace service: scaffold, repair, fsMap, placement edits
  ├─ Session manager: one ClaudeSDKClient per active project
  └─ SQLite persistence: projects, sessions, messages, events, checkpoints
  │
  ▼ Claude Agent SDK
Claude Code agent
  ├─ Reads/writes tscircuit source inside the project workspace
  ├─ Uses mounted `tscircuit` and `components` skills
  ├─ Runs allowed shell commands such as `tsci build`
  └─ Produces `dist/index/circuit.json`
```

The important design principle is that each project owns an isolated tscircuit workspace. The backend gives the agent that workspace as its current working directory, records the agent's outputs, and exposes workspace files to the browser as an fsMap. The frontend renders the workspace and sends user interactions, such as component drag placement, back to the backend for persistence.



VoltEdge system architecture

Figure: High-level VoltEdge architecture showing how the frontend, backend, agent runtime, and tscircuit workspace interact.

Implemented Features

Project and Workspace Management

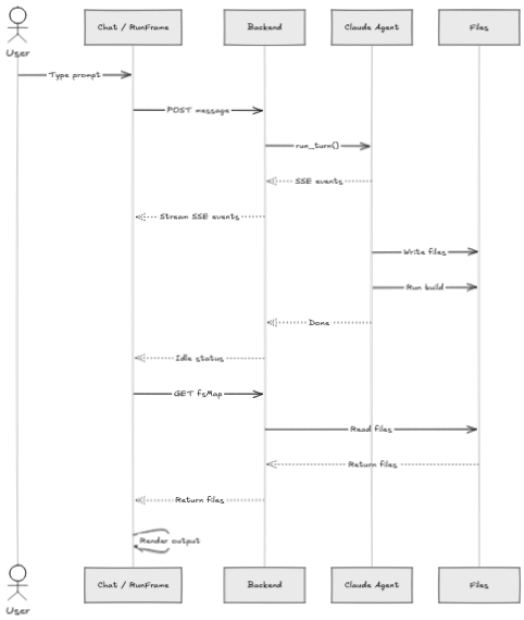
VoltEdge supports creating and listing projects through the backend API. Each new project receives a unique workspace directory under the configured workspaces path. The workspace service scaffolds a tscircuit project, mounts the necessary Claude skills, installs the reusable parts library, and ensures a clean `index.circuit.tsx` entrypoint exists.

The backend also supports:

- Project rename via `PATCH /projects/{project_id}`.
- Project deletion via `DELETE /projects/{project_id}`.
- Workspace self-healing when a project row exists but files are missing.
- fsMap retrieval through `GET /projects/{project_id}/fsmap`.

Conversational Agent Loop

The backend accepts user messages through `POST /projects/{project_id}/message`. A per-project lock prevents concurrent turns in the same project. The Session Manager records the user message, starts or reuses a Claude Agent SDK client, sends the query, and streams back SDK messages.



VoltEdge agent turn sequence

Figure: End-to-end sequence for a user prompt, including backend message handling, agent execution, build/checkpoint detection, SSE streaming, and frontend refresh.

The agent is configured with:

- `permission_mode="acceptEdits"` for workspace-local edits.
- A project-scoped working directory.
- Mounted `tscircuit` and `components` skills.
- A custom system prompt tuned for concise circuit-design behavior.
- A Bash allowlist for commands such as `tsci`, npm, bun, `git`, `ls`, `cat`, `grep`, `rg`, `find`, head, `tail`, `wc`, node, `mkdir`, pwd, and `echo`.
- Write/Edit confinement to the project workspace.
- `max_turns` from backend settings.

A key implementation detail is that Bash is deliberately not placed in `allowed_tools`. Phase 0 showed that doing so pre-approves Bash and bypasses `can_use_tool`. The current implementation keeps Bash gated through the callback.

SSE Event Streaming

The event mapper translates Claude Agent SDK messages into frontend events. Implemented event types include:

- `thinking`
- `assistant_text`
- `tool_use`
- `tool_result`
- `checkpoint`
- `error`
- done

The API client also recognizes planned event types such as `plan`, `question`, `build_status`, and `paused`, which prepares the frontend protocol for later phases.

The backend persists event history and exposes it through `GET /projects/{project_id}/events/history`, allowing the frontend to reload a previous session transcript.

Checkpoint Detection

The Session Manager detects build progress by watching the modification time of the generated circuit JSON artifact. When `dist/index/circuit.json` changes, the backend emits and persists a `checkpoint` event. The frontend responds by re-fetching the fsMap and causing the preview to update.

This gives the user a concrete signal that the agent produced a new build artifact and that the board preview should refresh.

Frontend User Interface

The frontend includes:

- A home page and app mode.
- Session sidebar.
- Chat panel.
- Message rendering.
- Thinking accordion.
- Tool call display.
- Resizable split layout.
- Preview pane.

The app subscribes to the backend SSE stream for the active project, appends events to the chat transcript, and refreshes the circuit workspace after checkpoint or done events.

Circuit Preview and Interactive Placement

The preview pane uses RunFrame from `@tscircuit/runframe/runner` and an inlined eval worker from `@tscircuit/eval/blob-url`. It evaluates the workspace source in the browser and provides schematic, PCB, and optionally CAD tabs depending on WebGL availability.

The UI supports drag-to-move behavior. When a user drags a PCB or schematic component, the frontend maps the edited element back to its source component name using the latest circuit JSON, then calls PUT `/projects/{project_id}/placement`. The backend rewrites pcbX/pcbY or schX/schY in `index.circuit.tsx` and returns the updated source.

This is an important usability improvement: user layout edits are not just visual; they are written back into the actual tscircuit source so later runs can preserve the user's placement choices.

Phase 0 Findings

Phase 0 was used to empirically de-risk the most important technical assumptions.

tscircuit Toolchain

The project verified that `tsci` version `0.0.2001` installs and can build a tscircuit project. It also established that bun is a hard runtime requirement because the `tsci` launcher uses a bun shebang.

Important findings:

- `tsci init -y` scaffolds the required workspace files.
- The scaffold writes `"tscircuit": "latest"`, so the workspace service must rewrite this to a locked version.

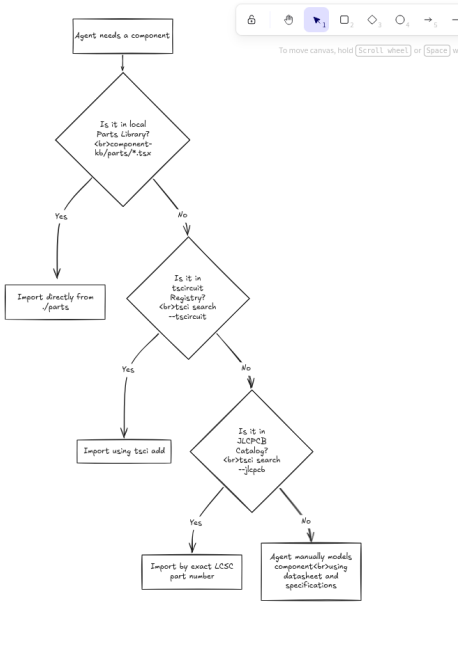
- `tsci build` outputs circuit JSON under `dist/<entrypoint>/circuit.json`, not directly under `dist/circuit.json`.
- The check chain includes netlist, schematic placement, placement, routing difficulty, trace length, and pin specification checks.

Non-Interactive Part Import

The team verified that exact JLCPCB/LCSC parts can be imported headlessly with:

```
tsci import C14877 --jlcpcb < /dev/null
```

This produced a component file with supplier part numbers, a footprint, pin labels, and 3D model URLs. This closed the risk that part import would require an interactive picker.



VoltEdge parts sourcing workflow

Figure: Planned sourcing workflow for moving from known reusable parts to exact JLCPCB/LCSC imports and, when necessary, hand-modeled fallback components.

Fabrication Export

The project verified that:

```
tsci export <file> -f gerbers -o out.zip
```

produces a complete fabrication archive containing Gerber layers, drill files, `bom.csv`, and `pick_and_place.csv`. This removed the need for a custom programmatic Gerber-generation fallback.

Browser Rendering

The render engine was validated through schematic, PCB, and GLTF export. The team also discovered that bundling `@tscircuit/runframe/runner` from source brings a large chain of undeclared dependencies and that React 19 is required by the viewer stack.

The architectural direction evolved from browser recompilation only toward using backend-built circuit JSON where possible. The current frontend implementation uses RunFrame for a richer in-browser editing experience.

Claude Agent SDK Smoke Test

A real Claude Agent SDK turn was run successfully. The SDK streamed system, thinking, assistant text, tool-use, tool-result, and done messages. The agent activated the tscircuit skill, wrote circuit source, ran `tsci build`, and produced `dist/index/circuit.json`.

The smoke test also showed a rough cost baseline of about `$0.10–$0.19` for simple turns, supporting the need for budget and turn guardrails.

Phase 1 Results

Phase 1 delivered the vertical slice: create a project, send one prompt, stream the agent's work, build a circuit artifact, and render/update the frontend.

Completed backend work includes:

- FastAPI app bootstrapping.
- SQLite/SQLModel models for projects, sessions, messages, checkpoints, and events.
- Workspace scaffold and fsMap service.
- Project create/list/rename/delete APIs.
- Message submission API.
- SSE event endpoint.
- Event history endpoint.
- Session Manager with one active ClaudeSDKClient per project.
- Agent SDK options and message mapping.
- Checkpoint detection after successful build artifact updates.
- Interrupt endpoint.
- Placement update endpoint.

Completed frontend work includes:

- Vite/React app layout.
- API client and EventSource wrapper.
- Chat stream rendering.
- Tool/thinking UI components.
- Session sidebar.
- Preview pane using tscircuit RunFrame.
- fsMap refresh on checkpoint/done.
- Drag placement persistence.
- WebGL detection for enabling CAD view only when supported.

The documented Phase 1 exit criteria passed locally: project creation, real agent turn, SSE streaming, generated circuit JSON, and continued operation after backend restart.

Current Technical Stack

Backend

- Python 3.14.4 was verified in Phase 0.
- FastAPI for HTTP routes.
- SSE-Starlette for server-sent events.
- SQLModel with SQLite persistence.
- Claude Agent SDK `>=0.2.110` for the agent runtime.

- Pydantic and pydantic-settings for schemas/configuration.
- AnyIO/asyncio for async coordination.

Frontend

- React 19.
- Vite.
- TypeScript.
- `@tscircuit/runframe`.
- tscircuit viewer packages.
- Radix UI packages.
- Tailwind CSS tooling.
- Supporting circuit and rendering packages required by the runframe/viewer ecosystem.

Circuit and System Tooling

- Node.js 22.22 verified.
- npm 10.9 verified.
- bun 1.3.14 verified.
- tscircuit `tsci` 0.0.2001 verified.
- git for repository and allowlisted agent operations.

Key Design Decisions

One Project, One Workspace

Every VoltEdge project maps to its own tscircuit workspace. This isolates generated files, local package state, imported parts, and build artifacts across user sessions.

Persistent Event History

Rather than treating SSE as ephemeral only, the backend persists events. This lets the frontend reconstruct prior conversations and tool activity when a project is reopened.

Agent Tool Guarding

The project explicitly guards shell access and file writes. Bash is allowed only through a command-prefix allowlist, and Write/Edit operations are confined to the project workspace.

Source as the Ground Truth

Even when the user drags components visually, the backend rewrites the source file. This keeps the tscircuit source authoritative and avoids a split between visual state and generated code.

Checkpoints Based on Build Artifacts

The system treats changes to `dist/index/circuit.json` as the signal for a meaningful circuit update. This is simple, concrete, and aligned with the rendering path.

Challenges Encountered

tscircuit Runtime Requirements

The `tsci` CLI depends on bun at runtime. This was not just a package-manager choice; it is required by the CLI launcher. The project had to account for bun in setup and agent PATH configuration.

Version Drift Risk

The initial design had a risk of two compilers: backend `tsci` and browser-side tscircuit evaluation. Phase 0 reduced this risk by identifying backend-built `circuit.json` as a reusable artifact and documenting version-pinning requirements for tscircuit and viewer packages.

runframe Dependency Complexity

The runframe runner path surfaced a long chain of dependencies that needed to be installed explicitly. The frontend package now contains many tscircuit, Radix, and utility packages to satisfy the viewer/runtime ecosystem.

Agent Permission Gotcha

A major finding was that listing Bash in `allowed_tools` bypasses `can_use_tool`. This would have weakened command allowlisting. The implementation now keeps Bash out of `allowed_tools` and gates it only through the callback.

Placement State Synchronization

The PCB viewer retains internal drag events, which can cause repeated delta application after multiple drags. The frontend addresses this by remounting RunFrame after a run that followed a drag, preventing stale edit-event replay from drifting component positions.

Remaining Work

The project is not yet complete. The following planned work remains:

Phase 2 — Full Agent Loop

- Structured `plan` events with parts, nets, board assumptions, and sourcing information.
- Hard-blocking `question` flow for important ambiguities.
- Plan approval endpoint and plan card UI.
- Build-status events showing check phases and violations.
- Guardrail counters for wall-clock time, tscircuit invocations, and token/cost limits.
- Paused/error banners for guardrail and failure states.
- Full PCB and 3D tab polish.
- Tiered sourcing workflow from registry to JLCPCB exact part to hand-modeled fallback.

Phase 3 — Export and Product Polish

- Source zip export.
- Fabrication zip export through `tsci export -f gerbers`.
- Clear manufacturability caveat in the export UI.
- Export buttons in the frontend.
- Session resume from stored Claude session IDs.
- Idle teardown sweeper for long-lived SDK clients.
- Project list polish.
- Configuration surface for caps, model policy, pins, and timeouts.

Cross-Cutting Work

- More backend unit tests.
- Toolchain smoke CI.
- Stubbed integration test for expected SSE sequence.
- Frontend component tests.
- Docker Compose or equivalent developer-environment setup.
- Final dependency pinning and lockfiles.

Testing and Verification Status

The repository contains backend tests for several important areas, including project CRUD, scaffold behavior, event history, placement, and scaffold error handling. The documentation also defines a broader testing strategy covering event mapping, guardrails, allowlists, fsMap reads, database CRUD, toolchain smoke tests, and full integration tests.

Empirical verification completed so far includes:

- tscircuit scaffold/build smoke checks.
- JLCPCB non-interactive import check.
- Gerber/BOM/PnP fabrication zip export check.
- Claude Agent SDK real-turn smoke test.
- Local Phase 1 vertical-slice execution.

Before presenting VoltEdge as complete, the project should still run and document a full current test pass and a manual browser verification of the UI.

Evaluation Against Goals

VoltEdge has successfully proven the feasibility of the core concept. The most important technical risk was whether an LLM coding agent could be safely connected to a real circuit-generation toolchain and produce previewable circuit artifacts in a web app. Phase 0 and Phase 1 show that this is possible.

The project currently satisfies these goals:

- Natural-language prompt can drive an agent turn.
- Agent can modify a real tscircuit workspace.
- Backend streams agent progress to the frontend.
- Generated circuit artifacts can be detected and surfaced as checkpoints.
- Frontend can load project history and render the circuit workspace.
- User visual placement edits can be persisted back into source.

The project does not yet fully satisfy these goals:

- Complete export workflow in the product UI.
- Fully structured planning and question/answer flow.
- Complete guardrail UX and budget enforcement display.
- Production-ready dependency pinning and CI.
- Complete manufacturability guidance and final design-review workflow.

Model and Data Card

This section documents the AI model/system behavior and the data handled by VoltEdge. VoltEdge does not train a new foundation model or a custom machine-learning model; it integrates an external agent model through the Claude Agent SDK and connects that model to a guarded tscircuit workspace.

Model Card

Field	Description
System name	VoltEdge 2.0 Agentic Circuit Design Assistant
Model type	Application-level AI assistant using the Claude Agent SDK with a tscircuit toolchain.
Training status	VoltEdge does not train or fine-tune a new model. The underlying Claude model is configured through backend settings.
Primary purpose	Convert natural-language circuit design requests into editable tscircuit projects with streamed progress and visual schematic/PCB previews.
Intended users	Students, makers, developers, and engineers prototyping simple circuit boards.
Inputs	User prompts, project workspace files, tscircuit/component library files, imported component metadata, and optional UI placement edits.
Outputs	tscircuit source such as `index.circuit.tsx`, build artifacts such as `circuit.json`, assistant messages, tool events, checkpoints, previews, and planned fabrication exports.
Deployment context	Local web application with a FastAPI backend, React frontend, Claude Agent SDK runtime, and tscircuit CLI/toolchain.

The model-facing agent is configured to work inside a project-scoped workspace. It is instructed to build exactly what the user asks for, prefer ready-made library components when available, keep layouts compact, run only necessary checks, and write the circuit to `index.circuit.tsx`, which is the file rendered by the UI.

Implemented safeguards include:

- One active agent turn per project.
- Project-scoped working directory.
- File Write/Edit confinement to the project workspace.
- Bash command allowlist rather than unrestricted shell access.
- Build-artifact checkpointing from generated `circuit.json` updates.
- Human review requirement before fabrication or real-world use.

Known limitations include:

- VoltEdge does not guarantee electrical correctness, safety, or manufacturability.
- The agent may choose incorrect parts, footprints, pin mappings, board dimensions, or routing assumptions.
- Ambiguous natural-language prompts can produce incomplete or incorrect designs.
- The system depends on the behavior and availability of external model and toolchain providers.
- Generated fabrication files must be reviewed by a qualified person before manufacturing.

Out-of-scope uses include safety-critical, medical, automotive, aerospace, high-voltage, or high-current electronics unless a qualified engineer independently reviews and validates the design.

Data Card

VoltEdge does not use a custom training dataset. The data handled by the system is application/session data generated during normal use.

Data category	Examples	Storage/handling
User prompts	Natural-language circuit requirements and follow-up instructions.	Stored as message records and sent to the configured agent provider during an agent turn.
Agent responses	Assistant text, thinking summaries where surfaced, tool-use summaries, and errors.	Streamed over SSE and persisted in event/message history.
Project metadata	Project ID, title, created timestamp, workspace path.	Stored in the local SQLite database.
Workspace files	`index.circuit.tsx`, package metadata, imported component files, tscircuit configuration, and markdown/source files.	Stored in project workspace directories.
Build artifacts	`dist/index/circuit.json`, generated previews, and planned export artifacts such as Gerber/BOM/PnP files.	Stored or read from project workspaces; checkpointed when build artifacts change.
Placement edits	`pcbX`, `pcbY`, `schX`, and `schY` coordinate changes from drag interactions.	Written back into `index.circuit.tsx` through the placement API.
External component metadata	Imported JLCPCB/LCSC part metadata, footprints, pin labels, and related package information.	Stored in generated/imported component files when used.

Privacy and security considerations:

- Users should not enter passwords, API keys, private credentials, or confidential designs unless the deployment environment and model-provider policy are trusted.
- Circuit designs, prompts, and workspace files may contain intellectual property.
- If the configured agent provider is external, prompts and relevant workspace context may be transmitted to that provider according to its API and retention policies.
- Local project/session data is stored in SQLite and filesystem workspaces; access controls depend on the deployment environment.

Quality and bias considerations:

- The system may favor components available in the local parts library, tscircuit registry, or JLCPCB/LCSC import path.
- Less common components or ambiguous module names may be modeled incorrectly.
- Availability of footprints and metadata can affect output quality.
- Generated boards should be validated with schematic review, pinout review, DRC/ERC checks where available, BOM verification, and datasheet inspection.

Recommended validation before fabrication:

- Run `tsci build` successfully.
- Inspect schematic and PCB previews.
- Verify all component pin mappings against datasheets.
- Review board dimensions, clearance, routing, and layer assumptions.
- Check BOM and pick-and-place files.
- Confirm fabrication outputs with the selected manufacturer.

Impact and Use Cases

VoltEdge is most useful as an early-stage circuit prototyping assistant. Potential use cases include:

- Quickly drafting simple breakout boards and sensor modules.
- Exploring board layouts from high-level descriptions.
- Teaching users how natural-language circuit requirements map to concrete components and nets.

- Generating editable tscircuit source as a starting point for further engineering work.
- Producing early fabrication artifacts once export polish is complete.

It should not be treated as a replacement for electrical engineering review. Fabrication exports must still be checked for correctness, sourcing, safety, power constraints, and manufacturability.

Conclusion

VoltEdge 2.0 demonstrates a practical path toward agent-assisted electronics design. The project combines conversational AI, guarded tool use, tscircuit compilation, persistent workspaces, and browser-based visual feedback into a coherent local prototype. The completed Phase 0 and Phase 1 work establishes the foundation: the toolchain works, the agent loop works, the backend can stream and persist progress, and the frontend can render and interact with generated boards.

The next step is to turn the vertical slice into a more complete product loop: structured planning, clarification questions, build-status visibility, export workflows, guardrails, session lifecycle management, and stronger automated testing. Once these are complete, VoltEdge will be much closer to a usable end-to-end assistant for rapid circuit-board prototyping.